Iris shader compatibility

FINDINGS, RISKS, AND STARTING SOLUTIONS / SALTS 1.1

A shared rendering backend comes first.

Current result: Iris 1.11.7 renders shader packs through OpenGL on Minecraft 26.3. Salts currently performs AA through Vulkan. Both mods can be installed, but active Iris shaders and Salts AA do not currently render together.

The backend split was reproduced in a real game, and confirmed in the released Iris JAR. Hiding a shader pack's AA controls cannot bridge that split. It is a separate user-interface step after the rendering integration exists.

What this report delivers

An audit of Iris internals, the current Salts integration, eight representative shader packs, nine successful game launches, the AA settings problem, and an implementation roadmap. Each problem includes a starting solution and a way to verify it.

Recommended direction

Build an OpenGL-compatible scene bridge first. Introduce spatial AA before temporal upscaling. Add explicit pack contracts for color, depth, jitter, motion, and denoising; investigate a GPU-only OpenGL/Vulkan bridge for the existing native upscalers. Treat frame generation as a later presentation project.

Scope: Research and disposable test instrumentation only. This task does not add production Iris compatibility or certify every existing or future shader pack.

Sources: [I01] Iris 1.11.7 Fabric 26.3 release; [I02] Iris backend selection plugin; [I03] Iris Vulkan placeholder

1. Evidence and scope

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Item	Inspected or tested
Repository	Current working tree; Minecraft 26.3, Fabric Loader 0.19.5, Fabric API 0.162.0+26.3; Salts 1.1. Prior uncommitted fixes preserved.
Iris release	Official Iris 1.11.7+mc26.3 Fabric JAR, Modrinth version vTN4NRGW. Download SHA-512 matched published metadata.
Iris source	Official 26.3 branch at 671a5c87747d. This is a branch snapshot, not a claim of exact release-source identity. Backend behavior was also checked in the release bytecode.
Sodium	0.9.2+mc26.3, the already qualified stable runtime. Iris declares Sodium 0.9.x in its released metadata.
Machine	Windows x64, RTX 3060 Ti, NVIDIA 591.86, Microsoft OpenJDK 25.0.2+10. 1280 x 720 controlled world captures.
Pack scope	Eight downloaded, hashed archives inspected; three pack families rendered. Paid, private, unavailable, or arbitrary future packs are not covered by these samples.

Read the evidence labels

Observed: a released build was run or captured. **Source:** supported by code or documentation. **Risk:** an inferred failure mode after a future bridge. **Proposal:** work to implement and validate, not an existing capability.

Nine successful game processes produced 15 capture groups and 30 recorded screenshot frames, excluding startup auto-captures. Two further no-Salts diagnostic controls produced four screenshots. One initial missing-Fabric-API staging attempt is excluded. All 11 qualified processes exited normally. These short probes establish compatibility boundaries; they are not exhaustive image-quality or performance certification.

Sources: [I01] Iris 1.11.7 Fabric 26.3 release

2. The reproduced blocker

RESEARCH / SOURCE AUDIT / IN-GAME PROBES



Observed: Iris's own Vulkan placeholder in the real game. Salts FXAA remained active. This is not an active shader-pack render.

Backend	Iris shader rendering	Salts AA
Vulkan	Placeholder screen offers OpenGL and a restart. A selected pack file alone does not mean it is rendered.	FXAA ran and produced captures.
OpenGL	Complementary rendered; Iris public API reported an active pack.	TAA request was rejected; effective mode stayed OFF.

Released bytecode contains the same backend-switch screen as the pinned source. IrisMixinPlugin chooses its mixin branch from options.txt before rendering starts. The preference, actual chosen device, and shader state must be checked separately; blindly forcing its OpenGL mixins onto Vulkan would leave direct GL calls and GL texture casts in the render path.

Sources: [I02] Iris backend selection plugin; [I03] Iris Vulkan placeholder

3. Where the render pipelines meet

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Stage	Iris pack pipeline	What Salts needs
Frame setup	Per-frame time, current/previous camera matrices; per-dimension pipeline.	One frame identifier and one temporal reset decision.
Shadows / prepare	Shadow geometry, custom images, compute, voxelization.	Do not accumulate shadow-pass cameras or count these as world frames.
Opaque world	Pack gbuffer outputs and depth; material-specific shaders.	Matching scene color, visible geometry, and motion.
Hand / translucent	Iris renders the hand itself and copies depth at distinct boundaries. Deferred and transparent phases follow.	Agreed hand policy, opaque-only color if needed, transparent masks.
Composite / final	Multiple raster/compute passes; pack temporal resolve, bloom, tone mapping, final output.	An explicit spatial or temporal insertion point, with known color semantics.
Game finish / HUD	Iris color-space processing follows level rendering.	AA must not accidentally process HUD; FG needs coherent HUD-free and presented images.

Salts currently redirects the vanilla main target, modifies the final world projection, captures vanilla opaque/transparency boundaries, and resolves native AA around GameRenderer.renderLevel. Those hooks are useful for vanilla and Sodium, but do not describe the meaning of an arbitrary pack's private render targets.

Do not hook a universal "composite5". Pack program numbers are author choices. One pack can tone-map there, another can trace lighting, and another can resolve TAA. Buffer contents can change meaning between passes.

Sources: [I06] Iris world pipeline; [I07] Composite passes and buffer flips; [I08] Final pass and target binding; [I09] Iris hand and frame hooks; [I10] Iris world and shadow hooks

4. Current mod audit: what is missing

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Current code location	Finding	Starting change
RenderRuntime; RenderBackendType	Only Vulkan is constructed. canUseAntiAliasing requires Vulkan.	Add an explicit OpenGL capability path, with per-method availability.
GameRendererMixin; ModernMinecraftHooks	Targets and jitter assume the present scene adapter. No active-Iris routing.	Introduce a scene-provider interface selected by actual renderer and active pack.
VulkanSceneDlssController / FsrController	Native image/view handles and command buffers are Vulkan.	Keep that implementation; add a separate GL or cross-API provider.
MotionVectorRenderer / geometry capture	Camera, entity, particle and moving-block support exists for the current pipeline.	Export velocities matching the pack's final visible geometry and clip convention.
LevelRendererMixin	Opaque snapshot and OIT paths refer to vanilla phases.	Define Iris-specific pre/post-translucent boundaries and material masks.
VulkanHandAaController; FG surface hooks	Hand isolation and generated presentation belong to the existing Vulkan flow.	Integrate Iris hand rendering and design GL presentation separately.
Optional compatibility modules	Sodium integration exists; no Iris scene/AA-option adapter was found.	Version-gated optional Iris module; preserve no-Iris class loading.

Source evidence is the current working tree, rather than a claim about the last committed revision. The built mod SHA-256 and the original configuration hashes are recorded in the evidence manifest. Production sources and the built JAR are kept unchanged by this audit.

5. FXAA, SMAA, NIS, and FSR1

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Spatial AA is the most practical first milestone

FXAA and SMAA: implement or port the existing full-screen passes through an OpenGL-compatible render provider. Consume the completed world image after the pack's final scene output and relevant scene color conversion, while excluding GUI. Specify input transfer function and orientation. SMAA also needs its lookup textures and correct display-size edge/search offsets.

This can have relatively broad coverage because it does not require object velocities. It still cannot undo a pack's prior TAA blur or recover detail already lost by an earlier upscale. Decide whether the selected policy intentionally permits spatial AA after pack-native TAA, rather than silently stacking both.

Spatial upscalers need genuine lower-resolution scene rendering

NIS and FSR1: feeding a full-resolution final image to an upscaler does not provide the expected rendering savings. Iris and the pack must agree on render size, viewport, scene-dependent buffers, and a display-sized output. Porting the kernel alone is insufficient. Apply sharpening once; the pack may already include a sharpen pass.

A pack can use fractional viewports inside full-size textures, fixed-size sky maps, tiled bloom, or special compute grids. Resizing every colortex indiscriminately will break these assumptions. The final program is a display-size output stage; that is not a universal low-resolution scene boundary.

Acceptance: compare pack-only output with the spatial bridge disabled; then test high-contrast edges, UI, gamma ramps, resizing, and sharpening halos. Confirm the low-resolution scene is actually cheaper before calling an upscaler supported.

Sources: [D06] Viewport, buffer sizes and blending; [D07] Final pass display-size output; [I08] Final pass and target binding

6. MSAA and SSAA need different strategies

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

MSAA cannot be added by multiplying one sample count

Source + risk: Iris builds ordinary GL_TEXTURE_2D color targets, including integer and packed attachments. Packs sample them as sampler2D, read them through image load/store, and retain history in ping-pong pairs. Multisampled attachments need a compatible topology and explicit resolves before those consumers. Resolving a material ID or a packed normal like ordinary scene color is not generally meaningful.

Starting solution: keep MSAA unavailable for active packs until a specific compatible forward/gbuffer path exists. For a qualified pack, multisample only approved geometry attachments, resolve color/depth/data with defined semantics, and retain single-sample histories. Validate every attachment's sample count and alpha-test behavior. MSAA on a fullscreen final quad does not remove world-edge aliasing.

SSAA is broader, but resolution semantics still matter

Risk: scaling the main window target does not automatically scale a pack's relative and fixed-size buffers correctly. Shadow resolution, voxel grids, exposure reductions, and sky lookups should not all scale with the world. More pixels can also exceed texture limits or VRAM budgets.

Starting solution: distinguish output size, world size, and each pack buffer's sizing rule. Render the approved world path larger, preserve fixed-size auxiliary resources, then downsample once before GUI. Adjust pixel-based effects and texture sampling to that world resolution. Native pack TAA can remain an explicit optional combination, subject to quality and cost tests.

Acceptance: gbuffer/depth correctness, foliage cutouts, clouds, integer material IDs, mixed resolutions, memory use, resize/reload, and clean output on unsupported packs. Report MSAA and SSAA separately in compatibility status.

Sources: [D02] Color attachments and flip semantics; [D06] Viewport, buffer sizes and blending; [I07] Composite passes and buffer flips

7. TAA, DLSS/DLAA, and FSR2/3

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Method	Inputs beyond scene color	Iris work required
Salts TAA	Correct history, jitter, matching depth, camera and object motion.	A single temporal owner; valid pack-space reprojection and resets.
DLSS SR / DLAA	SDK-compatible color/depth/motion, render/output extents, jitter, exposure interpretation and feature flags.	Native API bridge plus a pack-specific input contract. DLAA is native-size temporal reconstruction, not a spatial fallback.
FSR2 / FSR3 SR	Color/depth/motion, jitter, exposure, reactive and composition handling for best results.	Pack-correct translucency masks, resources and dispatch parameters.
FSR3 frame generation	Coherent real frames, depth/motion, frame timing, HUD-free inputs, safe presentation.	Additional presenter integration; an SR bridge does not automatically supply it.

The existing native bridges explicitly operate on Vulkan resources. They cannot consume an OpenGL texture name as a VkImage. A feasible cross-API prototype must share or copy GPU images through supported external-memory APIs, synchronize both queues, and match the physical adapter.

A late final-screen hook can be a fallback for some spatial filters. High-quality temporal upscaling generally needs a location before incompatible post effects, coupled to the right depth and motion. That location varies by pack. Do not treat one shader pack's successful DLSS image as evidence for all packs.

Starting scope: one known raster pack, one temporal method, frame generation OFF, fixed resolution, verified jitter and motion visualizations. Expand only after the input contract is demonstrated.

Sources: [N01] NVIDIA Streamline DLSS integration; [N02] Streamline device and resource integration; [A01] AMD FSR2 temporal integration; [A02] AMD frame interpolation integration

8. Shader-pack audit: common raster packs

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Inspected release	AA controls found in archive	Initial adapter policy
Complementary Reimagined r5.9.3; Unbound r5.9.3	lib/common.glsl: FXAA_DEFINE; TAA_DEFINE (-1/0/1); TAA_JITTER. -1 lets Detail Quality choose TAA.	Nominal AA-off mapping: TAA_DEFINE=0, FXAA_DEFINE=-1. Recheck compiled branches and quality dependencies. A reset to -1 can re-enable TAA.
BSL 10.1.8	lib/settings.glsl: TAA and FXAA booleans; TAA_MODE and TAA_SELECTIVE.	Disable actual TAA/FXAA toggles for temporal replacement; preserve the selective-entity policy only where still relevant.
Solas 3.8	lib/common.glsl: TAA and FXAA booleans, FXAA subpixel/sensitivity.	A boolean mapping is a starting point. Inspect final resolve, hand treatment and independent temporal effects before approval.

Important: Complementary's own option description associates temporal filtering with effect stability, quality, and sample-count savings. Disabling it can change both appearance and GPU cost. It is not safe to assume "TAA off" only removes edge smoothing.

Reimagined and Unbound share the inspected AA control names, but are separate configurations and render styles. The archive identity and profile must still be tracked separately. BSL and Solas were source-audited here, not run through the proposed Salts bridge.

References below identify exact published archives. Their complete hashes and the relative code paths/line locations are in pack-inventory.json. A filename or localized "Anti-Aliasing" label is not an adequate pack signature.

Sources: [P01] Complementary Shaders - Reimagined r5.9.3; [P02] Complementary Shaders - Unbound r5.9.3; [P03] BSL Shaders 10.1.8; [P08] Solas Shader 3.8

9. Shader-pack audit: temporal dependencies

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Inspected release	What makes it different	Starting solution
Photon v1.4	settings.glsl exposes TAA, FXAA and TAAU. TAAU depends on TAA; global.glsl changes scale only when both are defined. Separate AO, fog and cloud histories exist.	Coordinate TAA=false, FXAA=false, TAAU=false for a replacement prototype. Keep effect histories; export explicit scene/exposure and scale semantics.
MakeUp Ultra Fast 9.6	AA_TYPE has 0=off, 1=denoise, 2=TAA, 3=TAA plus FXAA. Some sky/auxiliary vertices still jitter for AA_TYPE>0.	Do not equate denoise-only with no jitter everywhere. Audit all passes before deciding whether mode 1 can coexist with external temporal AA.
Bliss 2.1.2	TAA and JITTER_STRENGTH controls. A zero value is not listed in the inspected jitter-strength option range.	Use a true compatible jitter path, rather than inventing an unsupported zero slider value. Source-only archetype here.
Rethinking Voxels r0.1-beta9	TAA_MODE is numeric; TEMPORAL_FILTER can also derive from reflection quality. Voxelized/ray-based lighting has separate dependencies.	Do not delete every temporal filter. Qualify lighting denoising and stochastic sampling independently from final AA.

Bliss is an older source sample whose current release metadata lists 26.3; it was not run here. This Rethinking Voxels archive does not list 26.3. Seven of the eight inspected archives list 26.3. These labels are author metadata, not Salts compatibility certification. Private path-traced packs need separate access and qualification.

Sources: [P04] Photon Shaders v1.4; [P05] Bliss Shaders 2.1.2; [P06] Rethinking Voxels r0.1-beta9; [P07] MakeUp - Ultra Fast 9.6

10. Can we disable and gray out pack AA?

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Yes for a known setting; no universal semantic detector exists

Source: Iris discovers pack-defined preprocessor options, numeric values, profiles, screens and translated labels. There is no common "AA enabled" key in the inspected API. Its public config interface toggles whole shaders, not arbitrary pack features. The UI uses custom option widgets rather than only Minecraft Button widgets.

Starting solution: add a version-gated optional Iris adapter, a pack fingerprint/profile registry, and an effective-options overlay applied before shader construction. Each approved profile identifies the real AA switches, associated jitter/upscale settings, valid values, and retained denoisers. Unknown packs should use native AA or a clearly labeled limited mode.

A disabled appearance must also block all mutation paths

Lock the identified option widgets visually and functionally. Cover left/right click, keyboard confirmation, reverse cycling, shift/control reset, numeric sliders, profile selection, imported option files, defaults, and queued updates on Apply. Reapply managed values after profile expansion and before compilation; one blocked button does not enforce the state.

Display a reason such as **Controlled by Salts: DLSS**, with the effective value visible. Keep unrelated shader controls usable. Do not lock anything merely because Iris is installed: activate this policy only when the chosen Salts method is actually running and the active pack/profile is compatible.

> **Do not rewrite the user's original ZIP.** Keep the user's settings separate from the managed effective overlay, restore their values when ownership changes, and rebuild safely if the pack fails compilation. This avoids permanent preset damage.

Sources: [D01] Shader-pack option definitions and profiles; [I04] Iris public API v0; [I05] Iris API configuration; [I11] Iris option widget input paths; [I12] Iris option loading and queues

11. One owner for temporal reconstruction

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Policy	Who controls jitter / final AA?	What remains available?
Pack native	The shader pack. Salts temporal resolve and scene scaling are inactive.	Native shader settings remain usable; qualified optional spatial sharpening can be explicit.
Salts spatial	The pack can retain its temporal denoising/AA if the profile explicitly permits it. Salts owns the selected late spatial pass.	Show combined filtering clearly and prevent accidental double sharpening.
Salts temporal	Salts owns final reconstruction, agreed jitter and render scale. Known conflicting pack AA/upscalers are overridden.	Independent GI/AO/cloud/SSR denoisers stay on where compatible.
Unqualified / failure	Fallback to pack native or disable the unsupported Salts method.	Explain the missing capability and preserve the saved Salts selection for later.

Make the owner state explicit in the AA menu: active pack identity, detected backend, compatible methods, and effective pack overrides. A shader-pack change must invalidate the profile, not silently reuse the preceding pack's buffer indices.

A quality profile can change sampling count and derived macros even when the visible AA label is unchanged. Apply user profile choices first, then calculate the managed options and compile once. If an override breaks the pack, remove it and return to native rendering with a clear status.

Why raw option values are insufficient

Observed in Complementary: after setting TAA_DEFINE=0, Iris's parsed option inventory still reported a TAA macro default of true. The source derives that macro inside conditional branches. Our probe's effectiveAaOptions field is an option-table readout, not a measurement of compiled execution. Validate preprocessed shader code and frame behavior before declaring AA disabled.

Sources: [D01] Shader-pack option definitions and profiles; [D08] Iris transformed shader debugging; [I12] Iris option loading and queues; [P01] Complementary Shaders - Reimagined r5.9.3

12. Motion vectors must follow shader geometry

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F04 - current captures are not a universal pack velocity buffer

Source + risk: Salts already reconstructs camera motion and captures moving entity, particle and block geometry. Shader packs can then displace vertices for wind, water, curvature, waving vegetation, or parallax-related depth effects. A velocity rendered from the original Minecraft vertex is wrong if the visible surface moved later in the pack shader.

Current/previous camera matrices provide static-world reprojection, not previous deformed geometry. A camera-only field cannot describe independently moving mobs, particles, vegetation, or clouds. Reflections and ray-generated content can move differently from their supporting surface.

Starting solutions, in order of fidelity

1. Author-exported motion: define a documented velocity output at a known stage, using current and previous final vertex positions. Include wind time, object pose, camera-relative origin and visibility. Keep the hand and translucent surfaces explicit.

2. Qualified shader transformation: for a known pack, add a previous-position path that reproduces its deformation at the previous time. Validate alpha testing, material variants and instanced draw IDs. An arbitrary GLSL rewrite is a maintenance-heavy fallback, not a universal guarantee.

3. Limited fallback: camera/depth reconstruction plus validated object overlays and conservative rejection masks. This can reduce damage but does not meet a claim of correct motion for all shader effects. Expose that limitation and retain pack-native reconstruction when it matters.

Acceptance: capture signed velocity and depth for a stationary camera, a camera pan, moving mobs, falling blocks, particles, wind-blown foliage, flowing water, shadows and distant geometry. Test correspondence at the final visible silhouette, not only a plausible colored mask.

Sources: [D04] Current and previous matrices; [N01] NVIDIA Streamline DLSS integration; [A01] AMD FSR2 temporal integration; [P04] Photon Shaders v1.4; [P03] BSL Shaders 10.1.8

13. Jitter and depth conventions

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F05 - exactly one agreed sampling pattern

Risk: Salts projection jitter plus the pack's gl_Position jitter can double-offset the world. Even if the screen looks stable at rest, motion and depth will be out of phase with the history. Jitter can differ by material, sky, hand, and upscale mode. Pixel units must refer to the actual input extent and viewport, with one sign/origin convention.

Starting solution: either disable the pack's final-AA jitter and inject Salts' sequence at all required pack stages, or explicitly consume the pack's known sequence and prevent Salts from adding another. Denoisers that rely on jitter must receive the agreed sample positions. Advance the sequence once per real world frame, not per shadow, portal view, GUI frame or generated frame.

Finding F06 - depth and matrix conventions change by phase

Source: Iris overrides clip-control reporting, transforms projection and clear behavior for pack rendering, and converts the exported gbuffer projection with MatrixUtils.undoRevZ. Its API also reports a non-shadow reverse-Z flag. These are distinct layers: one backend name or API flag does not prove a sampled texture's encoding. depthtex0/1/2 differ in transparency/hand inclusion; packs can also write gl_FragDepth or supply combined LOD depth.

Starting solution: annotate clip range, near/far, reversed/ordinary Z, clear value, infinite/finite projection, jitter status and depth extent. Convert only where required, then configure the native SDK consistently. Choose the depth layer matching the chosen color/hand policy. Reconstruct sky motion without treating the far clear value as nearby geometry.

Acceptance: visualize linear depth, UV reprojection and zero-motion jitter cancellation. Test camera cuts, third person, FOV changes, nausea/view bob, water, held items, and horizon/sky. MakeUp's AA_TYPE=1 still jitters some sky/auxiliary passes in the inspected code; a UI label is not proof that jitter is absent.

Sources: [D03] Depth buffer contents; [D04] Current and previous matrices; [I14] Iris clip-control override; [I16] Iris per-phase projection changes; [I17] Iris per-phase depth-clear changes; [I18] Iris gbuffer matrix conversions; [I19] Iris API depth-flag implementation; [N01] NVIDIA Streamline DLSS integration; [A01] AMD FSR2 temporal integration; [P07] MakeUp - Ultra Fast 9.6

14. Scene color, exposure, and post effects

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F07 - a floating-point texture does not define its meaning

Source + risk: colortex attachments are pack-owned storage. A given slot can contain linear scene radiance, exposure-scaled data, tone-mapped display color, packed normals, histories, or unrelated values at different stages. Photon's buffer declarations explicitly reuse attachments for different jobs. RGBA16F alone does not prove that a resource is linear HDR scene color.

Salts currently selects native color encoding for its own scene targets and performs controlled transfers. Iris introduces pack exposure, tone mapping, gamma and final color-space conversion. Incorrect placement can double-convert the signal, clip highlights, pump exposure, or produce haloing. Applying reconstruction after motion blur, depth of field or lens distortion also changes the data and motion relationship.

Starting solution

For each approved temporal profile, declare a scene-color producer, exact stage, buffer and flip side, transfer function, pre-exposure scale, exposure source, alpha meaning and output consumer. Insert reconstruction at that contract boundary. Leave each tone-map and display conversion in exactly one place. The SDK flags and any conversion shader must match the actual chosen signal.

Separate the temporal reconstruction branch from pack bloom/DoF/motion blur as appropriate for that pack. There is no guaranteed universal order: bloom may consume scene radiance, and other effects can mix back into the scene. A late display-color spatial pass can be a deliberately limited fallback, rather than masquerading as a full temporal integration.

Acceptance: HDR emissive blocks, sunrise/sunset, dark interiors, lightning, underwater transitions, exposure adaptation, bloom, DoF, motion blur and transfer-function ramps. Compare bypass output to the pack-only reference to detect accidental color processing changes.

Sources: [D02] Color attachments and flip semantics; [I06] Iris world pipeline; [I08] Final pass and target binding; [P04] Photon Shaders v1.4; [N01] NVIDIA Streamline DLSS integration; [A01] AMD FSR2 temporal integration

15. Transparency, reactive masks, and details

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F08 - vanilla opaque/translucent boundaries are insufficient

Risk: Salts' current opaque snapshot and material handling follow the vanilla/Sodium world sequence. Iris packs can render fog, water refraction, glass, particles and clouds in different buffers or compute passes. A late "final minus opaque" difference can also contain lighting and tone-map changes; treating all of it as transparency can destroy temporal stability.

Starting solution: capture opaque-only and completed color at compatible stages in the same color space. Prefer material/exported transparency coverage and composition masks over a screen-wide color heuristic. Record a dedicated contract for water/refraction, particle/emissive layers, weather and cloud history. Tune FSR reactive/composition behavior from those signals; follow the supported DLSS SDK's resource contracts separately rather than inventing an interchangeable mask API.

Finding F09 - texture-detail bias and alpha coverage are pack-specific

Source + risk: the previous Sodium integration adjusts known terrain sampling paths. An Iris-transformed shader, material atlas, normal/PBR lookup or shadow sampler may no longer match those patterns. A global negative LOD bias can sharpen noise, produce halos, or make cutout coverage shimmer. Applying a color-atlas policy to a packed-data texture is incorrect.

Starting solution: classify color/detail/material/shadow samplers. Apply render-scale bias only where appropriate, preserving explicit textureLod, texelFetch, derivative and shadow paths. Test alpha-tested geometry at the actual sample pattern. Disable a texture-detail patch when no known-safe target is available.

Acceptance: layered stained glass, water over glass, animated water normals, rain, smoke, enchanted items, cloud edges, leaves, fences, emissive textures and resource packs. Use temporal clips plus magnified crops; a static screenshot cannot establish that ghosting or shimmering is solved.

Sources: [D05] Program ordering and compute; [D06] Viewport, buffer sizes and blending; [A01] AMD FSR2 temporal integration; [P04] Photon Shaders v1.4; [I07] Composite passes and buffer flips

16. Hands, HUD, and frame generation

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F10 - Iris replaces the vanilla hand path

Source: Iris suppresses ordinary hand submission when a pack is in use and invokes its own solid/translucent hand rendering. The depth snapshots around the hand have distinct semantics. Salts' existing hand separation therefore cannot simply be assumed to intercept the same work.

Starting solution: choose a pack-compatible hand policy. Reconstruct the hand with matched hand motion/depth, or composite an independently rendered hand after reconstruction. Preserve shader lighting, transparency and reflections on held items. If removing the hand from history requires a fresh render, account for that cost explicitly.

Finding F11 - a Vulkan FG swapchain cannot present OpenGL automatically

Risk: the current FSR3 generation path owns Vulkan presentation resources and tracks real/generated presents. Iris's world image is OpenGL. An SR interop bridge only moves images; it does not turn the GL window's swap operation into the SDK's Vulkan swapchain. Incorrect ownership can create stalls, duplicate frames, pacing faults and broken overlays.

Starting solution: qualify ordinary reconstruction first. Then design a dedicated presenter, deciding which API owns the visible surface, how real frames and generated frames are scheduled, and how timing, HUD and hand layers are composed. Keep GUI, pause/menu behavior and screenshots coherent. Preserve F3's real-versus-presented count distinction.

Acceptance: inspect actual presented frames while swinging tools, eating, using transparent items, opening inventories/chat, changing shaders, resizing and alt-tabbing. In-game screenshots alone do not capture all generated presents. Current research did not run FG with an active Iris pack; the backend blocker prevents that.

Sources: [I09] Iris hand and frame hooks; [I10] Iris world and shadow hooks; [D03] Depth buffer contents; [A02] AMD frame interpolation integration

17. Lifetime, multi-view rendering, and safety

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F12 - pipeline and history identity must change together

Source: Iris caches per-dimension pipelines and destroys/recreates targets during reload. Texture IDs can be reused. Salts history must be keyed to a pipeline generation, pack/profile, view, extent and encoding; a raw GL handle alone is unsafe.

Starting solution: invalidate history and cached native bindings on pack reload, profile changes, dimension changes, camera cuts, resize, backend changes, world exit and failed compilation. Retire shared resources only after both APIs finish using them. Rebuild the scene contract before the next frame and restore pack-native rendering if reconstruction fails.

Finding F13 - shadow, portal, LOD and VR views are separate integrations

Risk: a moving shadow camera can overwrite the player's temporal state. Portals and VR can render multiple views with different matrices/extents in one display frame. Distant terrain can have its own depth and transforms. Shader packs and Iris extensions can change the drawing order further.

Starting solution: initially support the primary world view, with an explicit shadow-pass guard. Later allocate per-view histories and define combined depth/motion for supported LOD/portal/VR adapters. Test actual combinations rather than claiming that Iris support implies support for all of those mods.

Finding F14 - optional loading and mixin ordering

Starting solution: isolate Iris references behind optional class loading and a version gate. Prefer an upstream scene/lifecycle hook over competing redirects. Keep the compiler-pool workaround's deferred GPU finalization on the original scheduling, and test it with Iris's own PipelineBuilder wrapper. Preserve the current Sodium-only and no-Iris paths.

Sources: [I04] Iris public API v0; [I13] Dimension and pipeline lifetime; [I09] Iris hand and frame hooks; [I10] Iris world and shadow hooks

18. Architectural choices

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Approach	Benefit	Cost / boundary
A. OpenGL spatial provider first	Works in the backend Iris uses now; avoids changing shader-pack rendering wholesale.	New GL scene adapter and spatial passes. Temporal native methods still need further work. Recommended first implementation.
B. GL + Vulkan native-SR sidecar	Can reuse much of the existing native DLSS/FSR evaluation after GPU resource interop.	Own Vulkan device, external memory, compatible formats, cross-API synchronization, and pack input contracts. FG presentation is separate.
C. Extend or wait for Iris Vulkan	Longer-term common API could reuse more Salts infrastructure.	Iris's current GL pipeline, custom buffers, compute, transforms and state management need broad renderer work. Removing its backend guard is not a port.
D. Pack-native fallback	Preserves shader appearance for unqualified packs; useful immediately in a future compatibility UI.	Does not deliver Salts' selected AA method. State the fallback honestly.

Recommendation: stage A, then a narrowly scoped B prototype. Pursue C through a maintained upstream extension or explicitly maintained fork if its scope is accepted. Keep D throughout development and for unsupported pack capabilities.

"All Iris shaders" is a sensible coverage goal, but not a verifiable guarantee for arbitrary programs and hardware. Publish support per pack version, method and feature level. A shader that renders through Iris can still omit the data needed for correct external temporal reconstruction.

Sources: [I02] Iris backend selection plugin; [I06] Iris world pipeline; [K01] Khronos external memory and semaphores; [N02] Streamline device and resource integration

19. GPU interop: feasible, not implemented

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F15 - resource sharing must be designed explicitly

Observed: this RTX 3060 Ti OpenGL context reports GL_EXT_memory_object, GL_EXT_memory_object_win32, GL_EXT_semaphore and GL_EXT_semaphore_win32. This is a useful prerequisite check; no shared image or semaphore exchange was implemented or tested here.

Starting prototype: create exportable Vulkan allocations with compatible formats and usage; import their Windows handles into GL memory objects; expose GL textures backed by that memory. Copy/convert approved Iris inputs into those shared targets on the GPU. Submit Vulkan reconstruction only after a correctly signaled external semaphore, then signal completion for GL to consume the output.

Match the physical GPU using the available device identity/LUID information, not just a device name. Validate external-image format compatibility, tiling, dedicated-allocation requirements and handle lifetime. Do not cast a GL texture name to a Vulkan handle. Existing Iris-owned textures are not automatically exportable, so "GPU-only" does not imply "zero copy".

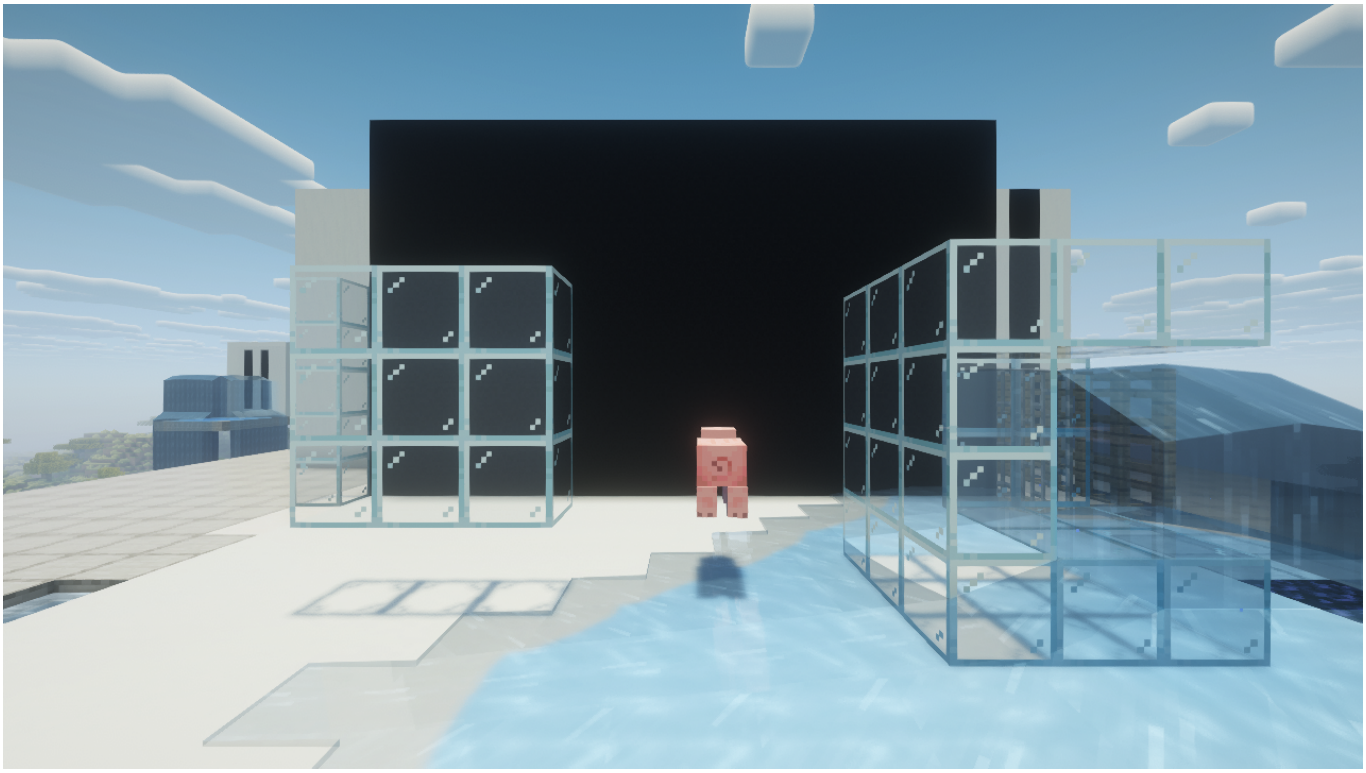
Use synchronization primitives actually supported across both APIs; do not assume Vulkan timeline semaphore semantics are available through the basic GL semaphore extension. Track layout/ownership transitions and multiple frames in flight. Avoid routine glFinish, vkDeviceWaitIdle or CPU image readback in the frame loop.

Prototype gate: first share a known test pattern in both directions, then depth and signed motion. Run GL debug and Vulkan synchronization validation; test at least AMD, NVIDIA and Intel where applicable. Only then attach the real pack scene and compare bypass output. Unsupported interop must leave the pack usable.

Sources: [K01] Khronos external memory and semaphores; [K02] Khronos Windows external handles; [N02] Streamline device and resource integration

20. Real-game reference images

RESEARCH / SOURCE AUDIT / IN-GAME PROBES



Observed: Complementary Reimagined running through Iris on OpenGL in the audit world. Salts AA is OFF because its backend gate is active.

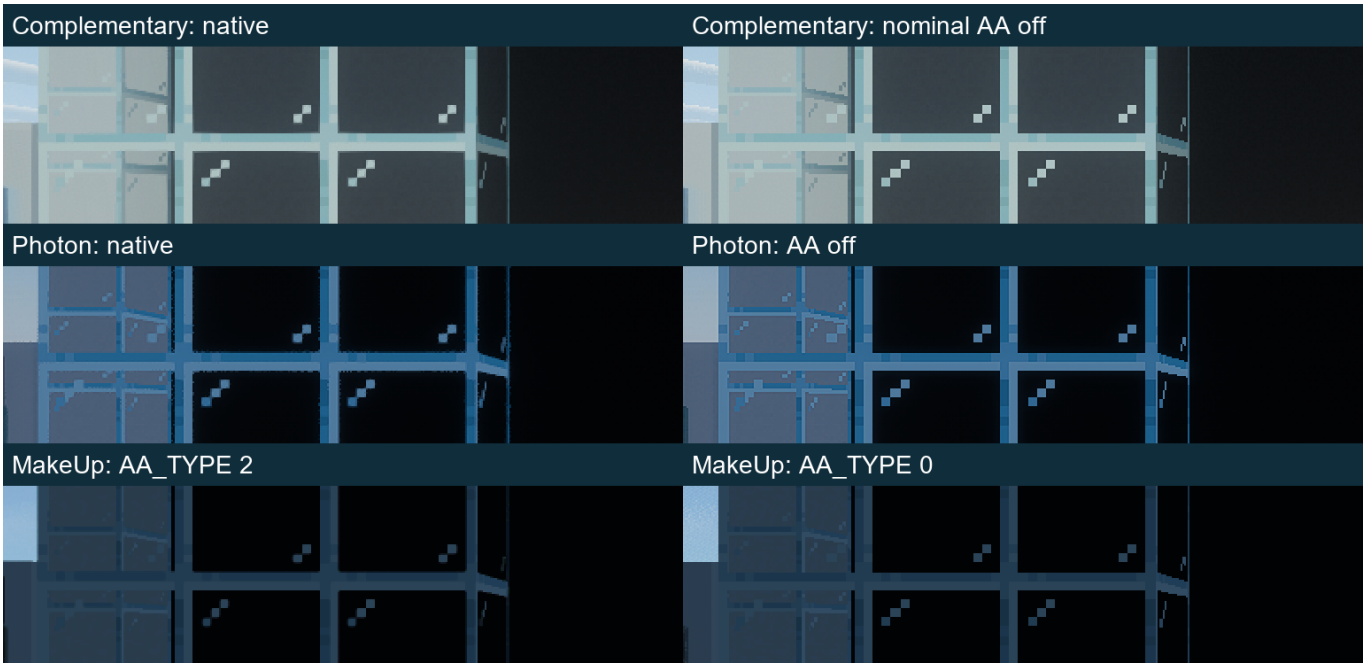
The same process rejected a Salts TAA request and kept its mode OFF. Iris's public isShaderPackInUse() remained true. This provides a shader-only reference; it is not evidence that Salts AA works on the pictured shader.

Completed baseline probes	Observed result
Complementary native and nominal AA-off	Both rendered and exited normally; TAA_DEFINE and FXAA_DEFINE option reads matched the requested values. Derived macro caveat applies.
Photon native and AA-off	Both rendered; TAA/FXAA/TAAU readouts matched requests.
MakeUp AA_TYPE 2 / 1 / 0	All three rendered; the requested numeric mode was reported by Iris.

Full-resolution screenshots and nearest-neighbor detail crops accompany the evidence. These captures verify active shader rendering and option ingestion; they do not establish motion quality for the unimplemented combined pipeline. Scene illumination and stochastic sampling can differ between fresh launches.

21. Detail comparison: disabling pack AA

RESEARCH / SOURCE AUDIT / IN-GAME PROBES



Paired 2x nearest-neighbor crops of the left glass/terrain region. All images are pack-only references with Salts AA OFF. Inspect the original PNGs at full resolution before making image-quality judgments.

Observed: edge smoothing and fine-detail appearance change when the pack AA options change. The world continues to render in these samples. This supports treating the options as real rendering controls, not merely decorative menu entries.

Limits: different fresh launches can vary clouds, lighting histories and noise. No temporal metric is inferred from these stills. The proposed Salts temporal replacement is absent, so these images neither prove nor disprove its future quality. Native AA-off may expose noise that a separate pack denoiser must retain or compensate for.

Future validation should record repeated fixed-seed camera paths and actual presented frame sequences, not rely on a single "before/after" still. Compare each method against an appropriate native pack reference at equal resolution and clearly stated sharpening.

Sources: [P01] Complementary Shaders - Reimagined r5.9.3; [P04] Photon Shaders v1.4; [P07] MakeUp - Ultra Fast 9.6

22. Findings register and starting fixes

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

ID / priority	Problem	First solution / validation gate
F01 / P0	No common active backend	GL scene provider first; prove active pack plus AA in the same frame.
F02 / P0	Wrong insertion point or target	Stage/flip-aware scene contract; a bypass must preserve the pack output.
F03 / P1	Unmanaged pack AA controls	Profile-based option overlay and all-input UI lock; restore settings.
F04 / P0	Velocity misses shader deformation	Export final-geometry motion; verify visible silhouette correspondence.
F05 / P0	Conflicting jitter / temporal owner	One sequence, one frame advance and coordinated denoisers.
F06 / P0	Wrong depth space or layer	Declared depth conventions and color-matching contents; reprojection tests.
F07 / P0	Color/exposure/post mismatch	Per-stage encoding and exposure contract; test highlight and adaptation ramps.
F08 / P1	Incorrect transparency signals	Pack-aware opaque/color/masks; test glass, particles, water and clouds.
F09 / P2	Unsafe texture-detail adjustments	Classify sampler roles; validate bias and cutout coverage.
F10 / P0	Hand does not follow vanilla path	Iris hand integration; inspect moving/translucent held items.
F11 / P0	FG cannot own GL presentation	Dedicated presenter and real/generated timing, after SR works.
F12 / P1	Stale targets/history on reload	Generation keys and deferred retirement across both APIs.
F13 / P1	Shadows / multiple views / LOD	Primary-view scope first, then per-view and combined-depth adapters.
F14 / P1	Optional loading and mixin conflict	Version-gated module; no-Iris and Sodium-only regressions.
F15 / P0	Missing safe GPU resource interop	Pattern/depth/motion sharing prototype with validation enabled.
F16 / P1	MSAA / SSAA resource topology	Per-method target rules; unsupported combinations stay unavailable.
F17 / P1	No arbitrary-pack semantic contract	Fingerprint registry plus author opt-in metadata; explicit fallback.
F18 / P2	MakeUp fixedTaaCorrection parse failure	No-Salts control reproduced it; component-wise expression prototype removed the diagnostic.

P0 = prerequisite for correctness of the affected method. P1 = required before claiming that combination supported. P2 = quality/diagnostic work. These priorities describe implementation dependencies, not an assertion that every inferred failure was reproduced.

23. Recommended implementation sequence

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Milestone	Concrete work	Completion gate
1. Detect and explain	Optional Iris module; actual backend/pack state; pack identity; method support status; preserve settings.	Iris installed but inactive does not disable valid Sodium AA. Active unsupported paths explain why.
2. Spatial support	OpenGL scene provider, frame boundary, FXAA/SMAA and single sharpening policy.	Active Iris pack + visible Salts spatial AA; no HUD filtering or bypass color changes.
3. One temporal pack	One maintained raster pack, one method, final geometry motion, depth/jitter/exposure contract, option ownership.	Debug buffers and controlled motion tests pass; pack AA really disabled at compiled/rendered level.
4. Native SR interop	GPU-only GL/Vulkan sharing; import supported user-supplied runtime; safe resource retirement.	DLAA and SR correctly reconstruct the pack scene, including transparency, without routine CPU stalls.
5. Broaden coverage	Additional pack profiles, TAA/FSR variants, resize/dimension/effect matrix, hardware validation.	Publish pack/version/method capability results; failed profiles fall back safely.
6. Advanced targets / FG	Qualified SSAA/MSAA paths, Iris hand policy, presenter and generated HUD behavior.	Inspect actual presented frames, timing, hand/HUD and long-session stability.

The ordering can change after prototypes. For example, a native-SR prototype can precede broad temporal pack support, but both gates must pass before shipping the combined feature. Do not make the UI appear fully compatible while the backend or scene contract is missing.

Preferred extension contract

A future Iris/pack extension should export scene-ready and frame-finished hooks, pack generation/view ID, typed color/depth/motion resources, jitter ownership, exposure, masks, resolution rules, and an effective-option policy. Prefer a small stable contract to many fragile injections into numbered composite passes. This is a proposal; it is absent from the inspected public API.

Sources: [I04] Iris public API v0; [I05] Iris API configuration

24. Validation plan before a support claim

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Area	Required cases and evidence
Pack coverage	Complementary styles, BSL, Photon, MakeUp, Solas, volumetric-heavy packs, and separately qualified ray/path-traced packs. Pin exact archives and profiles.
Temporal geometry	Static camera convergence; slow/fast pans; disocclusion; strafing; entities; particles; animated foliage; flowing water; moving blocks; sky. Save color, depth, motion, masks and jitter metadata.
Effects	Bloom, exposure, DoF, motion blur, SSR, volumetric clouds/fog, emissive/PBR textures, rain, underwater, cutouts, translucent items, alternate dimensions.
Resolution / mode	720p, 1080p, 1440p, 4K where feasible; native/DLAA and all supported SR qualities; model switches; window resize, fullscreen and GUI scale.
Lifecycle	Pack on/off/switch; profile/import/reset; failed compile; resource reload; backend preference changes/restart; world exit/rejoin; dimensions; alt-tab; device loss where supported.
UI ownership	Mouse, keyboard, sliders, reverse cycle, resets, imported options and profiles cannot bypass a managed lock. Switching owner restores player preferences.
Presentation	Real/presented frame accounting; captured generated presents; hand/HUD/menu behavior; pacing and latency; no duplicate/stale frames.
Regression / performance	No Iris, installed inactive Iris, Sodium-only; GPU and CPU timings, memory growth and allocation churn; no routine CPU readbacks or device-idle stalls.

Pass criteria: first require correct input correspondence, no GPU validation faults, no stale resources and an unchanged bypass reference. Then assess ghost trails, edge wobble, disocclusion recovery, color, noise and sharpening with controlled motion clips and magnified crops. Report limitations per method; do not collapse every image-quality result into a single "compatible" flag.

This audit only performed the baseline/backend and option-ingestion subset. It did not run an implemented Iris temporal bridge, cross-API memory exchange, combined FG, multi-GPU tests or all shader packs.

25. An isolated pack-parser problem

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Finding F18 - MakeUp fixedTaaCorrection fails to resolve

Observed: MakeUp 9.6 with AA_TYPE=2 and AA_TYPE=1 produced an Iris custom-uniform resolution warning for fixedTaaCorrection. The expression at shaders/shaders.properties line 188 multiplies a vec2 made from pixelSizeX and pixelSizeY by 0.51. AA_TYPE=0 did not show that warning in its qualified run.

This matters to the research because the correction is consumed by selective-TAA vertex paths. The fact that the world still renders does not establish that every temporal correction was bound correctly. A working shader menu and a true active-pack API result are therefore necessary but incomplete compatibility checks.

Negative control: the warning is not caused by Salts

A separate OpenGL instance containing Iris, Sodium, Fabric API and minimal capture instrumentation, with **no Salts JAR**, reproduced the fixedTaaCorrection warning. It rendered two screenshots, reported an active pack and exited normally. The earlier compiler-pool workaround was absent from this control too. This isolates the warning to the pack/Iris combination in this environment.

Tested starting solution

In a private disposable copy of the pack, the same expression was rewritten to construct the vector from two scalar products: **vec2(pixelSizeX * 0.51, pixelSizeY * 0.51)**. A second no-Salts run rendered normally and the fixedTaaCorrection resolution warning was absent.

> **Limit:** this demonstrates a successful parser workaround, not a completed rendering fix in the production mod. An implementation should validate the resulting uniform values and selective-TAA geometry, then use an approved pack/profile overlay or an upstream pack change. Original downloaded shader archives remain unchanged and are not redistributed.

Sources: [I15] Iris custom uniform resolution; [P07] MakeUp - Ultra Fast 9.6

26. Source index: Iris and shader interface

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Sources were accessed on 25 October 2026. Iris code links pin the inspected branch snapshot. The release JAR was independently inspected for its backend gate. Documentation under /current/ can change; use the pinned source and saved evidence when implementing.

Sources: [I01] Iris 1.11.7 Fabric 26.3 release

Sources: [I02] Iris backend selection plugin

Sources: [I03] Iris Vulkan placeholder

Sources: [I04] Iris public API v0

Sources: [I05] Iris API configuration

Sources: [I06] Iris world pipeline

Sources: [I07] Composite passes and buffer flips

Sources: [I08] Final pass and target binding

Sources: [I09] Iris hand and frame hooks

Sources: [I10] Iris world and shadow hooks

Sources: [I11] Iris option widget input paths

Sources: [I12] Iris option loading and queues

Sources: [I13] Dimension and pipeline lifetime

Sources: [I14] Iris clip-control override

Sources: [I15] Iris custom uniform resolution

Sources: [I16] Iris per-phase projection changes

Sources: [I17] Iris per-phase depth-clear changes

Sources: [I18] Iris gbuffer matrix conversions

Sources: [I19] Iris API depth-flag implementation

Documented shader-pack interface

Sources: [D01] Shader-pack option definitions and profiles

Sources: [D02] Color attachments and flip semantics

Sources: [D03] Depth buffer contents

Sources: [D04] Current and previous matrices

Sources: [D05] Program ordering and compute

Sources: [D06] Viewport, buffer sizes and blending

Sources: [D07] Final pass display-size output

Sources: [D08] Iris transformed shader debugging

27. Source index: SDKs and exact pack samples

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

Sources: [N01] NVIDIA Streamline DLSS integration

Sources: [N02] Streamline device and resource integration

Sources: [A01] AMD FSR2 temporal integration

Sources: [A02] AMD frame interpolation integration

Sources: [K01] Khronos external memory and semaphores

Sources: [K02] Khronos Windows external handles

Pack archives inspected

Sources: [P01] Complementary Shaders - Reimagined r5.9.3

Sources: [P02] Complementary Shaders - Unbound r5.9.3

Sources: [P03] BSL Shaders 10.1.8

Sources: [P04] Photon Shaders v1.4

Sources: [P05] Bliss Shaders 2.1.2

Sources: [P06] Rethinking Voxels r0.1-beta9

Sources: [P07] MakeUp - Ultra Fast 9.6

Sources: [P08] Solas Shader 3.8

The pack release pages identify the author-distributed archives. pack-inventory.json records each exact download URL, SHA-512, version, release date, declared Minecraft versions, license metadata and AA option locations. Shader source was inspected in local research scratch space; shader ZIPs and NVIDIA DLLs are not redistributed with this report.

Local repository files supporting the code comparison

RenderRuntime.java; RenderBackendType.java; GameRendererMixin.java; LevelRendererMixin.java; VulkanSceneDlssController.java; VulkanSceneFsrController.java; VulkanMotionVectorRenderer.java; VulkanEntityMotionRenderer.java; VulkanHandAaController.java; VulkanTextureDetail.java; native/dlss_bridge/src/salts_dlss_bridge.cpp; native/fsr_bridge/src/salts_fsr_bridge.cpp. Paths are relative to src/client/java/org/betterLostItems/salts_anti_aliasing unless otherwise stated. Exact repository-relative paths and hashes are in the evidence manifest.

28. Reproduction and remaining limits

RESEARCH / SOURCE AUDIT / IN-GAME PROBES

How to reproduce the key result

1. Use an isolated Minecraft 26.3 Fabric instance with Sodium 0.9.2, Iris 1.11.7 and the current Salts JAR. Keep backend preference and launch arguments consistent, then restart after changes.

2. Select Vulkan. Salts FXAA can render, but opening Iris displays the backend-switch placeholder. A shaderpack ZIP in the folder is not evidence of shader rendering.

3. Select OpenGL and restart. Enable Complementary Reimagined r5.9.3. Verify the active-pack API state. A Salts TAA request leaves the effective mode OFF and reports the Vulkan requirement.

4. For the AA-control probes, apply the exact per-pack values described in sections 8-9, reload/restart, record Iris's parsed option values, and inspect transformed code plus temporal output before claiming the entire algorithm is disabled. Never infer final execution from a derived macro's default alone.

What is not established

No active Iris + Salts temporal reconstruction exists in this build, so this report cannot show its DLSS/FSR image quality. No GPU interop exchange or GL FG presenter was implemented. Thirty stills across short baseline probes do not measure long-session stability, every effect, hardware diversity or all possible packs.

The tests use an offline development identity. Authentication/Realms messages and Windows performance-counter errors appear in logs and are not shader rendering failures. Pack option-menu/identifier warnings were also observed. MakeUp's custom-uniform resolution diagnostic was reproduced without Salts and a private expression workaround was tested, as described in section 25.

Conclusion: broad Iris coverage is an achievable engineering direction, but it requires a common rendering path, explicit scene data and pack-aware temporal control. The first useful patch should establish those boundaries, rather than only hide AA buttons.